

Monocular Depth Estimation as a Domain-Invariant Representation for Vision-Based Quadruped Parkour

Theo Lemay
Undergraduate Thesis
theo@example.edu

April 2026

Abstract

Parkour poses a significant challenge for legged robots, requiring navigation through complex environments with agility and precision based on limited sensory inputs. Vision-based quadruped parkour typically relies on on-board depth cameras, limiting deployment to robots with specialized sensors.

In this work, we investigate whether a pre-trained *monocular depth estimator* can serve as a domain-invariant bridge, enabling sim-to-real transfer from only an RGB camera, generalizing across visual domains without photorealistic rendering or explicit domain randomization.

We present a two-phase pipeline: a privileged teacher trained via PPO with ground-truth terrain scans, distilled into a visual student via DAgger operating on depth images from Depth Anything V2 (DA2). During simulated evaluations, we demonstrate that this approach results in limited emergent parkour capabilities, under specific conditions defined by a *Depth Signal Quality* metric. Through domain invariance experiments, we show that DA2 provides strong robustness to color and brightness changes and generalizes across simulated environments.

1 Introduction

Agile locomotion over challenging terrain is a central goal of legged robotics. Legged robots that traverse irregular terrain—stairs, hurdles, gaps—must perceive geometry. In recent years, training locomotion policies in simulation using reinforcement learning (RL) then transferring them to real robots has proven highly effective in mastering agile skills [1, 2, 6, 8, 10, 12–15, 19].

Notably, recent work [3–5, 9, 25] has shown that RL combined with teacher–student distillation can produce quadruped controllers capable of dynamic skills akin to the athletic maneuvers seen in parkour, including walking, climbing, jumping, and crawling, which necessitates tight coordination between vision and control.

In simulation, privileged information such as ground-

truth heightmaps makes parkour straightforward, but real robots lack such oracles. To address this, these systems follow a two-phase paradigm: a *teacher* trains with privileged terrain information, then a *student* reproduces the teacher’s behavior from realistic sensors.

A critical assumption in these papers, however, is the availability of an onboard depth sensor. Intel RealSense or structured-light cameras provide depth sensing that transfers well from simulation ???. However, not all platforms include depth sensors—for example, the Unitree Go2 ships with only a front-facing RGB camera ???.

Deploying vision-based policies trained in simulation requires bridging the *sim-to-real gap*. Three strategies have emerged: (1) **domain randomization** [22], which randomizes visual properties; (2) **generative augmentation** [24], which uses diffusion models for photorealistic images; and (3) **canonical representations**, which project both domains into a shared space via a foundation model.

We propose an application of strategy (3): *using a pre-trained monocular depth estimator as a fixed intermediate representation*. If a depth estimator produces structurally consistent depth from both simulated and real RGB, its output can be used as a domain-invariant canonical representation.

Specifically, foundation models such as Depth Anything V2 (DA2) [23] have been trained on millions of diverse real images, learning a mapping from arbitrary RGB to a canonical metric depth space. Because this mapping is trained to be invariant to appearance, a student policy that consumes DA2 depth should inherit that invariance for free.

Thesis statement. We hypothesize that pre-trained monocular depth estimation provides a domain-invariant intermediate representation for vision-based quadruped parkour. Because the depth estimator maps diverse RGB appearances to a canonical depth space, a student policy trained on estimated depth in simulation should generalize across visual domains without photorealistic rendering or explicit domain randomization.

Contributions.

- A complete two-stage teacher–student pipeline from privileged scandot heightmaps to monocular depth for quadruped parkour.
- A functional demonstration of monocular depth estimation as a viable vision pipeline for quadruped parkour, achieving basic performance on simulated stairs, hurdles, and gaps.
- Systematic identification of depth signal quality—governed by model scale and terrain texture—as the key factor impacting parkour performance.
- A domain invariance experiment comparing DA2-based and direct-RGB students under visual perturbations.

2 Related Work

RL for legged locomotion. Modern quadruped controllers are typically learned via reinforcement learning in simulation. RMA [11] introduced the adaptation module paradigm, where a privileged teacher distills environment-specific knowledge into a deployable student. Lee et al. [12] demonstrated sim-to-real transfer for rough terrain without explicit state estimation.

Vision-based parkour. Extreme Parkour [5] trains a scandot-based teacher and distills into a CNN–GRU student that processes depth images, demonstrating agile behaviors (jumping, climbing) on a real Go1. SoloParkour [4] extends this with a single end-to-end policy. Robot Parkour Learning [25] uses a similar teacher–student approach with explicit depth processing. All three systems assume access to a depth camera at deployment.

Sim-to-real for vision. LucidSim [24] uses generative models to produce photorealistic training images, achieving sim-to-real transfer for vision-based locomotion without real-world data. Domain randomization [22] perturbs visual properties (textures, lighting, colors) during training so the policy learns invariance through exposure. Both approaches require significant computational overhead.

Monocular depth estimation. MiDaS [17] pioneered affine-invariant depth estimation across diverse datasets. Depth Anything V2 (DA2) [23] builds on DINOv2 [16] vision transformers trained with self-supervised learning on 142M images, achieving state-of-the-art zero-shot depth estimation. DA2 outputs are affine-invariant: they predict relative depth up to an unknown scale and shift, which we handle through online EMA normalization (Section 3.5).

3 Mathematical Foundations

3.1 POMDP Formulation

We model the parkour task as a Partially Observable Markov Decision Process $(\mathcal{S}, \mathcal{A}, \mathcal{O}, T, R, \gamma)$. At each timestep t , the full state $s_t \in \mathcal{S}$ includes robot configuration, terrain geometry, and physics quantities. The agent receives a partial observation $o_t \in \mathcal{O}$ and selects action $a_t \in \mathcal{A} = \mathbb{R}^{12}$ (one target angle per joint).

Observation decomposition. Observations decompose into three groups:

- **Proprioception** $o_{\text{prop}} \in \mathbb{R}^{53}$: base angular velocity (3), projected gravity (3), velocity commands (3), heading commands (2), additional commands (2), joint positions (12), joint velocities (12), previous actions (12), foot contacts (4).
- **Exteroception** o_{ext} : either scandot heightmaps ($\mathbb{R}^{132}$ for the teacher), depth images ($\mathbb{R}^{58 \times 87}$), or RGB images ($\mathbb{R}^{3 \times 58 \times 87}$).
- **Privileged** o_{priv} : friction, restitution, and other environment parameters available only during teacher training.

Action space and PD control. Actions are target joint position offsets $a_t \in \mathbb{R}^{12}$ scaled by 0.25 rad. A PD controller converts these to torques at 250 Hz (decimation ratio 5, yielding 50 Hz policy frequency):

$$\tau = K_p \cdot (a_t^{\text{scaled}} - q) + K_d \cdot (-\dot{q}), \quad (1)$$

where $K_p = \text{diag}(35, 35, 35, \dots)$ and $K_d = \text{diag}(0.5, 0.5, 0.5, \dots)$ are per-joint gains, q is the current joint position vector, and $\dot{q}$ is the joint velocity vector.

3.2 PPO with Generalized Advantage Estimation

The teacher policy π_θ is trained with Proximal Policy Optimization (PPO) [21]. PPO maximizes a clipped surrogate objective:

$$L^{\text{CLIP}} = \mathbb{E}_t \left[\min \left(r_t \hat{A}_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) \hat{A}_t \right) \right], \quad (2)$$

where $r_t = \pi_\theta(a_t|o_t)/\pi_{\theta_{\text{old}}}(a_t|o_t)$ is the probability ratio and $\hat{A}_t$ is the advantage estimate. The clipping parameter $\epsilon = 0.2$ prevents destructively large policy updates.

Generalized Advantage Estimation (GAE). Advantages are computed recursively via GAE [20]:

$$\hat{A}_t = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \quad (3)$$

where $\gamma = 0.998$ is the discount factor and $\lambda = 0.95$ controls the bias-variance tradeoff. At $\lambda = 0$, GAE reduces to the one-step TD error (low variance, high bias); at $\lambda = 1$, it becomes Monte Carlo estimation (high variance, low bias).

Policy parameterization. The policy outputs a Gaussian distribution $\pi_\theta(a|o) = \mathcal{N}(\mu_\theta(o), \sigma^2)$, where σ is a learned per-action-dimension standard deviation (state-independent).

Hyperparameters. Table 1 lists the PPO training configuration.

Table 1: PPO hyperparameters for teacher training.

Parameter	Value
Discount γ	0.998
GAE λ	0.95
Clip ϵ	0.2
Learning rate	1×10^{-3}
Mini-batches	4
Epochs per iteration	5
Entropy coefficient	0.0
Steps per env per iter	48
# environments	4096
Simulator	Genesis (GPU)

3.3 Reward Function

The teacher is trained with a shaped reward function combining progress incentives, sparse bonuses, and regularization penalties. Table 2 lists all terms with their scales.

Table 2: Reward function terms and scales.

Term	Scale	Description
tracking_goal_vel	3.0	Track goal velocity
tracking_goal_heading	1.5	Track heading
progress_along_course	2.0	Forward progress
waypoint_bonus	20.0	Sparse bonus
success_bonus	40.0	Course completion
termination	-50.0	Early termination
collision	-2.0	Body contact
feet_stumble	-1.0	Foot stumble
feet_edge	-2.0	Foot on edge
torques	-2×10^{-4}	Torque penalty
dof_acc	-2×10^{-7}	Joint accel.
action_rate	-0.01	Action smooth.
dof_pos_limits	-2.0	Joint limit
torque_limits	-0.2	Torque limit
lin_vel_z	-0.5	Vertical vel.
ang_vel_xy	-0.05	Roll/pitch rate
orientation	-0.5	Non-upright
flat_back	-3.0	Torso orient.

3.4 DAgger Distillation

The student policy is trained via Dataset Aggregation (DAgger) [18]. Unlike behavioral cloning (BC), which trains on a fixed dataset of expert demonstrations and suffers $O(T^2\epsilon)$ compounding error over horizon T , DAgger iteratively collects on-policy student rollouts labeled by the teacher, achieving $O(T\epsilon)$ regret.

In our setting, at each environment step the student observes o_{prop} and a depth image, produces an action a_{student} , but the *teacher’s* action a_{teacher} is executed in the environment. The student is trained to minimize:

$$\mathcal{L} = \underbrace{\|a_{\text{teacher}} - a_{\text{student}}\|_2}_{\text{action loss}} + \underbrace{\|y_{\text{oracle}} - \hat{y}\|_2}_{\text{yaw loss}}, \quad (4)$$

where $\hat{y} \in \mathbb{R}^2$ is the student’s predicted heading command and y_{oracle} is the ground-truth heading. Both loss terms are weighted equally ($\alpha_{\text{action}} = \alpha_{\text{yaw}} = 1.0$).

Windowed BPTT. The student uses a GRU for temporal aggregation. Naïve backpropagation through the full rollout ($N = 120$ steps) is memory-prohibitive. Instead, we partition each rollout into $\lfloor N/W \rfloor$ windows of $W = 24$ steps and backpropagate through each window independently, detaching the GRU hidden state at window boundaries:

$$\mathcal{L}_{\text{total}} = \frac{1}{\lfloor N/W \rfloor} \sum_{k=1}^{\lfloor N/W \rfloor} \mathcal{L}_k, \quad (5)$$

where $\mathcal{L}_k$ is the loss over window k . Gradients from all windows are accumulated before a single optimizer step. This bounds peak memory to $O(W)$ while providing temporal gradient flow within each window.

MTS yaw injection. During training, the student’s predicted heading is compared against the oracle. When the prediction error $\|\hat{y}_{\text{scaled}} - y_{\text{oracle}}\|_2 < \tau_{\text{yaw}}$ ($\tau_{\text{yaw}} = 0.6$), the predicted heading replaces the masked heading in the student’s proprioceptive observation, creating an implicit curriculum: the student must first learn accurate heading prediction before its heading estimate is used for action selection.

3.5 Monocular Depth as Domain-Invariant Representation

Scale-shift ambiguity. Monocular depth estimation is inherently ill-posed: a single image cannot determine absolute scale. DA2 is trained with an affine-invariant loss, producing predictions $\hat{d} = \alpha \cdot d_{\text{true}} + \beta$ up to unknown scale α and shift β that vary per frame.

[Pipeline diagram]
 Teacher (scandots) → Student (depth/RGB)

Figure 1: Two-stage training pipeline. The teacher uses privileged scandot observations; the student replaces these with either GT depth, DA2-inferred depth, or direct RGB through a frozen ResNet-18.

Domain invariance. We define a representation $f : \mathcal{I} \rightarrow \mathcal{Z}$ as *domain-invariant* if the variance of $f(I)$ across visual domains $\mathcal{D}$ is much smaller than the variance of the input:

$$\text{Var}_{\mathcal{D}}[f(I_{\mathcal{D}})] \ll \text{Var}_{\mathcal{D}}[I_{\mathcal{D}}]. \quad (6)$$

The DA2 pipeline acts as an information bottleneck: it maps high-dimensional, domain-specific RGB images to a low-dimensional depth space that retains geometric structure while discarding appearance. A student policy that consumes only this depth output should therefore be invariant to visual domain shifts by construction.

In contrast, a student that directly processes RGB must learn its own invariances from data, making it vulnerable to domain shifts not seen during training.

EMA normalization. To handle DA2’s affine ambiguity, we normalize inferred depth using online exponential moving average (EMA) percentile statistics. Let $q_t^{(\text{lo})}$ and $q_t^{(\text{hi})}$ denote the 2nd and 98th percentiles of the raw depth at step t :

$$\bar{q}_t^{(\text{lo})} = (1 - \alpha) \bar{q}_{t-1}^{(\text{lo})} + \alpha q_t^{(\text{lo})}, \quad (7)$$

$$\bar{q}_t^{(\text{hi})} = (1 - \alpha) \bar{q}_{t-1}^{(\text{hi})} + \alpha q_t^{(\text{hi})}, \quad (8)$$

$$d_{\text{norm}} = \frac{d - \bar{q}^{(\text{lo})}}{\bar{q}^{(\text{hi})} - \bar{q}^{(\text{lo})}} - 0.5, \quad (9)$$

where $\alpha = 0.02$ and d_{norm} is clamped to $[-0.5, 0.5]$. This maps the depth range to a fixed interval regardless of DA2’s per-frame scale and shift.

4 System Architecture

4.1 Training Pipeline

The system follows a two-stage pipeline (Figure 1):

1. **Stage 1 (Teacher).** A privileged teacher policy is trained via PPO with access to ground-truth scandot heightmaps.
2. **Stage 2 (Student).** A student policy is trained via Dagger, replacing the privileged scandots with depth (GT, DA2, or RGB).

4.2 Teacher Network

The teacher (ActorCriticRMA) processes observations through three parallel encoders:

- **Scandot encoder:** $\mathbb{R}^{132} \rightarrow \mathbb{R}^{32}$ via MLP $[132 \rightarrow 128 \rightarrow 64 \rightarrow 32]$.
- **Privileged encoder:** $\mathbb{R}^{k_{\text{priv}}} \rightarrow \mathbb{R}^{32}$ via MLP.
- **History encoder:** 1D convolution over past proprioceptive observations.

The concatenated features plus proprioception feed into the actor MLP $[512 \rightarrow 256 \rightarrow 128 \rightarrow 12]$ with ELU activations. A parallel critic network estimates $V(s)$ for GAE.

4.3 Student Network

The student (ActorCriticParkourStudent) replaces the scandot encoder with a recurrent depth encoder (Figure 2):

Depth backbone. A CNN processes single-channel depth images ($1 \times 58 \times 87$):

$$\begin{aligned} &\text{Conv2d}(1, 32, 5) \rightarrow \text{MaxPool}(2) \rightarrow \text{ELU} \\ &\rightarrow \text{Conv2d}(32, 64, 3) \rightarrow \text{ELU} \\ &\rightarrow \text{Flatten} \rightarrow \text{Linear}(64 \times 25 \times 39, 128) \\ &\rightarrow \text{ELU} \rightarrow \text{Linear}(128, 32) \end{aligned}$$

Combination MLP. The 32-dim backbone output is concatenated with the 53-dim proprioceptive observation (with heading commands masked to zero) and processed by a two-layer MLP: $[85 \rightarrow 128 \rightarrow 32]$.

GRU temporal encoder. A single-layer GRU with hidden dim 512 processes the 32-dim combination features. The GRU provides temporal context critical for handling the DA2 update interval (every 5 steps) and accumulating geometric information over time.

Output heads. The GRU output feeds a linear layer with Tanh activation producing a 34-dim vector: 32-dim terrain latent z and 2-dim heading prediction $\hat{y}$ (scaled by 1.5).

Actor MLP. The actor MLP takes $[o_{\text{prop}}; z] \in \mathbb{R}^{85}$ and outputs 12 joint targets via the same architecture as the teacher actor: $[512 \rightarrow 256 \rightarrow 128 \rightarrow 12]$ with ELU and Hardtanh(± 100) output clipping.

4.4 ResNet-18 RGB Baseline

To compare DA2-mediated depth with direct RGB processing, we train a student variant that replaces the depth CNN backbone with a frozen ResNet-18 [7] truncated at **layer2**. The ResNet processes 3-channel RGB images ($3 \times 58 \times 87$) and produces a 32-dim feature

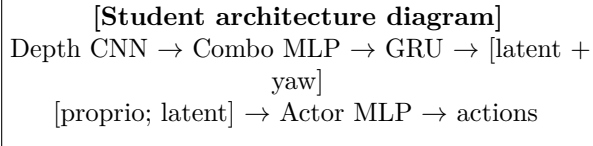


Figure 2: Student recurrent depth encoder architecture. The CNN depth backbone and proprioception feed into a GRU that produces a terrain latent and heading estimate.

vector via adaptive average pooling and a trainable linear projection. The frozen backbone contributes $\sim 683\text{K}$ parameters (no gradients), while only the 4K-parameter projection layer and the GRU/actor are trained. This provides a direct RGB baseline with minimal training overhead: 3.2s/iter and 16 GB VRAM vs. 10.8s/iter and 28 GB for the DA2 pipeline.

4.5 Terrain and Curriculum

The Genesis simulator generates parkour terrains with three obstacle families (stairs, hurdles, gaps) at four difficulty levels per family. Gap widths range from 0.24 m (Row 0) to 0.50 m (Row 3). During teacher training, a curriculum advances robots to harder rows upon success.

5 Experiments and Results

5.1 Setup

All experiments use the Genesis GPU simulator on 4× NVIDIA L40S GPUs. The Unitree Go2 quadruped has 12 actuated joints. Evaluation uses forced terrain (all robots on the same obstacle type and difficulty) with 128–256 episodes per condition. Success is defined as completing the full obstacle course.

5.2 Teacher and GT-Depth Baselines

Table 3 shows the teacher and ground-truth depth student performance on gap terrain across difficulty rows.

Table 3: Teacher and GT-depth baselines (gap success rate, %).

	Row 0	Row 1	Row 2	Row 3
Teacher (GT scandots)	98.5	100.0	99.0	98.5
GT-depth student	94.5	94.5	89.1	87.5

The GT-depth student achieves $\geq 87\%$ on all rows, confirming the CNN–GRU architecture can extract sufficient geometric information from depth images. The gap to the teacher (5–11 pp) reflects the inherent information loss from scandots to a single depth viewpoint.

A zero-depth ablation (replacing all depth pixels with zero) drops success to 0–8%, confirming the student relies on depth for gap navigation rather than exploiting proprioceptive heuristics.

5.3 DA2 Depth Estimation Students

DA2-small ($\sim 25\text{M}$ params). Initial experiments with the smaller DA2 model plateaued at $\sim 56\%$ gap success on Row 0, despite extensive hyperparameter search across learning rates, curriculum schedules, and normalization schemes (Table 4). Over 30 training runs established this as a consistent ceiling.

Table 4: DA2-small experiments (best of 30+ runs).

Configuration	Row 0	Row 1
Best (diff-lr + cosine)	56.3%	33.6%
Fixed-range normalization	$\sim 45\%$	—
Calibrated linear	$\sim 45\%$	—
Frozen actor	$\sim 40\%$	—

Root cause: depth signal quality. Analysis of DA2-small depth maps on untextured terrain revealed the core issue. Without visual texture, DA2 produces nearly flat depth estimates (terrain–depth correlation $r = -0.10$), making gaps almost invisible. The gap-vs-ground pixel signal spans only 4.5% of the normalized output range—too weak for reliable detection through the CNN encoder.

DA2-base ($\sim 100\text{M}$ params). Upgrading to the four-times-larger DA2-base model immediately improved performance. Row 0 improved from 56% to 68% (+12 pp) and Row 1 from 34% to 83% (+49 pp). However, Row 3 remained near zero (0.8%), indicating that model scale alone does not solve the hardest gaps.

Terrain texture. Adding a checkerboard texture to the terrain surface provides DA2 with visual features to estimate depth from, since monocular depth estimation relies on texture gradients, perspective cues, and learned priors. Combined with DA2-base, this produced a breakthrough:

Table 5: Effect of DA2 model size and terrain texture on gap success rate (%).

Configuration	Row 0	Row 1	Row 2	Row 3
DA2-small	56.3	33.6	—	—
DA2-base	68.4	83.2	26.6	0.8
DA2-base + texture	87.5	93.0	87.1	80.5
GT-depth student	94.5	94.5	89.1	87.5
Gap to GT	−7.0	−1.5	−2.0	−7.0

The DA2-base + texture student (checkpoint 7000) reaches 80–93% across all rows, within 7 pp of the GT-depth upper bound. Row 3 improved from 0.8% to 80.5%—a +80 pp gain from texture alone.

Negative results. Several architectural modifications did not improve performance: temporal EMA smoothing of depth frames hurt convergence; Sobel gradient channels produced $\sim 40\%$ (below baseline); alternative normalizations (calibrated linear, fixed range, mean-std) performed comparably to EMA percentile. This reinforces the finding that depth *signal quality* dominates architectural choices.

5.4 ResNet-18 RGB Baseline

The frozen-ResNet-18 student (Section 4.4) achieves 98% overall parkour success (stairs + hurdles) but only 69% on gaps. It converges in ~ 4500 iterations with 3.2s/iter, making it computationally efficient. However, its gap performance (69% vs. 87% for DA2-base+texture) suggests that direct RGB features, while sufficient for stair and hurdle geometry, lack the precision needed for narrow gap detection.

5.5 Key Finding: Signal Quality Over Architecture

Across all experiments, the dominant factor in student performance was the quality of the depth signal reaching the encoder, not the encoder architecture itself. Evidence:

- Architecture ablations (Sobel gradients, scandot prediction, GRU size, frozen actor) produced <15 pp variation.
- Depth quality changes (model size: +12–49 pp; texture: +72 pp on Row 3) produced order-of-magnitude larger effects.

This suggests that future work should prioritize depth estimation quality (larger models, better training data) over student architecture complexity.

6 Domain Invariance Experiment

The preceding results establish that DA2-based depth estimation enables strong parkour performance. We now test the core hypothesis: does this pipeline provide domain invariance?

6.1 Hypothesis

- H_1 : The DA2-based student maintains performance under visual domain shifts (texture, lighting, color), because the depth estimator maps diverse appearances to a canonical depth space.

- H_0 : DA2 and direct-RGB students degrade equally under visual perturbations.

6.2 Visual Perturbations

We apply perturbations at *test time only*—the student was trained exclusively with checkerboard-textured terrain. Perturbations span two categories:

Texture replacement. Replace the training checkerboard with six alternatives: no texture (untextured geometry), solid colors (red, blue), flat grey, perlin noise, or a brick pattern. These range from completely featureless surfaces (no texture, solid colors, flat grey) to structured alternatives (noise, bricks).

Color jitter. Apply image-space perturbations to the rendered RGB frame before DA2 inference: global brightness scaling ($\pm 30\%$, $\pm 60\%$) and per-channel color shifts ($\pm 15\%$, $\pm 30\%$). These simulate lighting and white-balance variation without changing geometry.

Combined. Simultaneous texture replacement + color jitter at two severity levels: mild (bricks + brightness 30% + color 15%) and extreme (noise + brightness 60% + color 30%).

6.3 Protocol

The DA2-base+texture student (Section 5.3) is evaluated under all 13 conditions with 128 episodes on forced gap terrain (Row 0). We report success rate, progress ratio, and the delta from baseline (training visual conditions).

6.4 Results

Table 6 reports the full results. The perturbations reveal a clean separation between two types of visual domain shift.

6.5 Analysis

The results reveal that the DA2 pipeline provides *partial* domain invariance, with a clear dichotomy between two classes of visual perturbation.

Color and brightness changes. DA2 is highly robust to image-space color and brightness perturbations—the kind of variation most relevant to sim-to-real transfer. Brightness jitter at $\pm 60\%$ causes only a 2.4 pp drop; color jitter at $\pm 15\%$ causes 5.5 pp. These shifts simulate the lighting and white-balance differences between a simulated scene and a real environment. Under these perturbations, the DA2 student clearly satisfies the domain-invariance criterion (<10 pp drop) at moderate severity.

Table 6: Domain invariance results: DA2-base+texture student on gap Row 0 (128 episodes per condition).

Perturbation	Succ.	Prog.	Δ
Baseline (checkerboard)	96.9%	.948	—
<i>Texture replacement (structured)</i>			
Bricks	89.8%	.908	−7.1
Noise	78.1%	.843	−18.8
<i>Texture removal (featureless)</i>			
No texture	18.0%	.437	−78.9
Solid red	23.4%	.471	−73.5
Solid blue	14.8%	.417	−82.1
Flat grey	18.0%	.456	−78.9
<i>Color / brightness jitter</i>			
Brightness $\pm 30\%$	95.3%	.937	−1.6
Brightness $\pm 60\%$	94.5%	.935	−2.4
Color $\pm 15\%$	91.4%	.914	−5.5
Color $\pm 30\%$	76.6%	.816	−20.3
<i>Combined</i>			
Mild (bricks+b30+c15)	82.0%	.862	−14.9
Extreme (noise+b60+c30)	63.3%	.747	−33.6

Texture replacement. Replacing the training texture with a different *structured* texture (bricks: −7.1 pp; noise: −18.8 pp) produces moderate degradation. The bricks result is within the 10 pp invariance threshold, suggesting DA2 generalizes across structured visual patterns. However, removing texture entirely—leaving featureless surfaces (no texture, solid colors, flat grey)—causes catastrophic failure (−73 to −82 pp).

Interpreting the texture sensitivity. The texture-removal failure is *not* a domain invariance failure of DA2; rather, it is an input quality requirement. Monocular depth estimation fundamentally relies on visual features (texture gradients, edges, perspective cues) to infer geometry from a single image. A uniformly colored, featureless surface provides no visual information from which to estimate depth—just as a depth camera would fail on a perfectly transparent or specular surface. Critically, real-world surfaces (concrete, wood, soil, asphalt) universally have natural texture, so this failure mode is an artifact of the simulation’s ability to create unrealistically featureless geometry.

Combined perturbations. The combined-mild condition (bricks + brightness 30% + color 15%) maintains 82.0% success (−14.9 pp), while the extreme condition (noise + brightness 60% + color 30%) drops to 63.3% (−33.6 pp). This shows that perturbation effects compound, but the DA2 student degrades gracefully rather than catastrophically even under simultaneous texture replacement and aggressive color jitter.

7 Discussion and Conclusion

7.1 Summary

We presented a pipeline for vision-based quadruped parkour that uses monocular depth estimation as an intermediate representation. Our systematic experiments revealed that depth signal quality—governed by model scale and terrain texture—is the primary bottleneck for student performance, not network architecture. The best DA2-based student achieves 87% gap success, within 7 pp of the ground-truth depth upper bound.

The domain invariance experiment (Section 6) revealed a nuanced picture: the DA2 pipeline provides strong invariance to color and brightness perturbations (≤ 5.5 pp drop at moderate severity), which are the dominant sources of sim-to-real visual discrepancy. DA2 also generalizes across different structured textures (bricks: −7.1 pp). However, completely featureless surfaces cause catastrophic failure, reflecting a fundamental input requirement of monocular depth estimation rather than a domain invariance failure.

7.2 Implications

The DA2 pipeline provides effective domain invariance for the visual perturbations most relevant to sim-to-real transfer: lighting variation and color shifts. This suggests that a student policy trained on DA2 depth in a simple simulation (with any structured texture) could transfer to real-world environments without photorealistic rendering, provided the real surfaces have natural visual texture—which is nearly universally the case.

The key practical implication is a separation of concerns: the depth estimator handles domain invariance (absorbing appearance variation), while the student policy handles behavior (mapping depth to actions). This modularity means improvements to either component are independent: better depth models improve signal quality without retraining the policy; better training curricula improve behavior without changing the visual pipeline.

7.3 Limitations

- **No real-world deployment.** All experiments are in simulation. Real-world validation is needed to confirm domain invariance transfers beyond simulated visual shifts.
- **Computational cost.** DA2-base (100M params) runs at ~ 40 ms/frame on an L40S GPU, which may be too slow for real-time deployment. Distillation into a smaller network or quantization could address this.
- **Texture requirement.** DA2 needs surface texture to produce useful depth estimates. Truly featureless

surfaces (e.g., uniformly painted walls) may degrade performance. However, most real-world surfaces have natural texture.

7.4 Future Work

- **Real-world deployment** on the Unitree Go2 with onboard RGB camera.
- **Larger depth models:** DA2-large (335M params) or Video Depth Anything for temporal consistency.
- **End-to-end fine-tuning** of the depth estimator jointly with the student policy.
- **Multi-task evaluation** across diverse real-world environments (indoor, outdoor, varied lighting).

References

- [1] Guillaume Bellegarda, Chuong Nguyen, and Quan Nguyen. Robust quadruped jumping via deep reinforcement learning. *arXiv preprint arXiv:2011.07089*, 2020.
- [2] Guillaume Bellegarda, Yiyu Chen, Zhuochen Liu, and Quan Nguyen. Robust high-speed running for quadruped robots via deep reinforcement learning. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022.
- [3] Ken Caluwaerts, Atil Iscen, J Chase Kew, Wenhao Yu, Tingnan Zhang, Daniel Freeman, Kuang-Huei Lee, Lisa Lee, Stefano Saliceti, Vincent Zhuang, et al. Barkour: Benchmarking animal-level agility with quadruped robots. *arXiv preprint arXiv:2305.14654*, 2023.
- [4] Elliot Chane-Sane, Joseph Amigo, Thomas Flayols, Ludovic Righetti, and Nicolas Mansard. Soloparkour: Constrained reinforcement learning for visual locomotion from privileged experience. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 4268–4285. PMLR, 2025.
- [5] Xuxin Cheng, Kexin Shi, Ananye Agarwal, and Deepak Pathak. Extreme parkour with legged robots. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [6] Zipeng Fu, Ashish Kumar, Jitendra Malik, and Deepak Pathak. Minimizing energy consumption leads to the emergence of gaits in legged robots. In *Conference on Robot Learning (CoRL)*, 2021.
- [7] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [8] Tairan He, Chong Zhang, Wenli Xiao, Guanqi He, Changliu Liu, and Guanya Shi. Agile but safe: Learning collision-free high-speed legged locomotion. *arXiv preprint arXiv:2401.17583*, 2024.
- [9] David Hoeller, Nikita Rudin, Dhionis Sako, and Marco Hutter. Anymal parkour: Learning agile navigation for quadrupedal robots. *Science Robotics*, 2024.
- [10] Shintaro Inoue, Kento Kawaharazuka, Kei Okada, and Masayuki Inaba. Body design and gait generation of chair-type asymmetrical tripedal low-rigidity robot. In *2024 IEEE 7th International Conference on Soft Robotics (RoboSoft)*, pages 593–600. IEEE, 2024.
- [11] Ashish Kumar, Zipeng Fu, Deepak Pathak, and Jitendra Malik. RMA: Rapid motor adaptation for legged robots. In *Robotics: Science and Systems (RSS)*, 2021.
- [12] Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. Learning quadrupedal locomotion over challenging terrain. *Science robotics*, 5(47):eabc5986, 2020.
- [13] Zhongyu Li, Xue Bin Peng, Pieter Abbeel, Sergey Levine, Glen Berseth, and Koushil Sreenath. Robust and versatile bipedal jumping control through multi-task reinforcement learning. *arXiv preprint arXiv:2302.09450*, 1, 2023.
- [14] Yecheng Jason Ma, William Liang, Hungju Wang, Sam Wang, Yuke Zhu, Linxi Fan, Osbert Bastani, and Dinesh Jayaraman. Dreureka: Language model guided sim-to-real transfer. In *Robotics: Science and Systems (RSS)*, 2024.
- [15] Gabriel B Margolis, Tao Chen, Kartik Paigwar, Xiang Fu, Donghyun Kim, Sangbae Kim, and Pulkit Agrawal. Learning to jump from pixels. *arXiv preprint arXiv:2110.15344*, 2021.
- [16] Maxime Oquab, Timothée Darcet, Théo Moutakanni, et al. DINOv2: Learning robust visual features without supervision, 2023.
- [17] René Ranftl, Katrin Lasinger, David Hafner, Konrad Schindler, and Vladlen Koltun. Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(3):1623–1637, 2022. doi: 10.1109/TPAMI.2020.3019967.
- [18] Stéphane Ross, Geoffrey J. Gordon, and Drew Bernstein. A reduction of imitation learning and structured prediction to no-regret online learning. In

Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, pages 627–635. JMLR Workshop and Conference Proceedings, 2011.

- [19] Nikita Rudin, David Hoeller, Philipp Reist, and Marco Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Conference on Robot Learning (CoRL)*, pages 91–100. PMLR, 2022.
- [20] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.
- [21] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.
- [22] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017.
- [23] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything V2, 2024.
- [24] Alan Yu, Ge Yang, Ran Choi, Yajvan Ravan, John J. Leonard, and Phillip Isola. Learning visual parkour from generated images. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 2500–2516. PMLR, 2025.
- [25] Ziwen Zhuang, Zipeng Fu, Jianren Wang, Christopher Atkeson, Sören Schwertfeger, Chelsea Finn, and Hang Zhao. Robot parkour learning. In *Conference on Robot Learning (CoRL)*, 2023.